Latest updates: https://dl.acm.org/doi/10.1145/3769765

PDF Download
3769765.pdf
11 March 2026
Total Citations: 0
Total Downloads: 524

Published: 05 December 2025

Citation in BibTeX format

RESEARCH-ARTICLE

Beyond Vector Search: Querying With and Without Predicates

JIADONG XIE, Chinese University of Hong Kong, Hong Kong, Hong Kong

JEFFREY XU YU, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong, China

SIYI TENG, Chinese University of Hong Kong, Hong Kong, Hong Kong

YINGFAN LIU, Xidian University, Xi'an, Shaanxi, China

Open Access Support provided by:

Xidian University

Chinese University of Hong Kong

The Hong Kong University of Science and Technology (Guangzhou)

Beyond Vector Search: Querying With and Without Predicates

JIADONG XIE, The Chinese University of Hong Kong, China
JEFFREY XU YU, The Hong Kong University of Science and Technology (Guangzhou), China
SIYI TENG, The Chinese University of Hong Kong, China
YINGFAN LIU*, Xidian University, China

k-ANN search has been extensively studied to find k approximate nearest neighbors for a given query vector in a high-dimensional dataset, where a data item is represented as a vector. As there are many new emerging real-world applications that have categorical/numerical attributes associated with vectors, it is highly needed to support k-ANN search with additional predicates on such attributes. In this paper, we study k-ANN queries, $q = (v_q, c_q)$, where v_q is a query vector and c_q is a predicate on categorical/numerical attributes. Note that the conventional k-ANN search is a k-ANN query when $c_q = \emptyset$. In the literature, some can support the cases when $c_q = \emptyset$, some can support the cases when c_q is on categorical attributes, and some can support the cases when c_q is on numerical attributes. But none of them can support all cases efficiently. In this paper, we propose an all-in-one approach. Our approach supports conventional k-ANN search in the same way as the state-of-the-art approaches, and supports the predicates in a similar or even better way compared to the approaches that are tailored to support either categorical attributes or numerical attributes. We conduct extensive performance studies and confirm the accuracy and the efficiency of our approach in comparison with the state-of-the-art approaches.

CCS Concepts: • **Information systems → Information retrieval query processing**.

Additional Key Words and Phrases: Approximate Nearest Neighbors Search, High-Dimensional Vector

ACM Reference Format:
Jiadong Xie, Jeffrey Xu Yu, Siyi Teng, and Yingfan Liu. 2025. Beyond Vector Search: Querying With and Without Predicates. *Proc. ACM Manag. Data* 3, 6 (SIGMOD), Article 300 (December 2025), 26 pages. https://doi.org/10.1145/3769765

1 Introduction

In the current landscape of big data and AI, effectively managing high-dimensional vectors is essential for contemporary applications like information retrieval [23], recommendation systems [28], and large pre-trained language models [19]. Machine learning models transform various data types into vector representations, empowering real-world applications to perform queries within vector spaces, such as text [27], images [29], audio [5]. Let $V = \{v_1, v_2, \cdots\}$ be a set of vectors. A fundamental problem is k-ANN search, which is to find k approximate nearest neighbors in V for a given query vector v_q. However, numerous real-world applications increasingly leverage

*Yingfan Liu is the corresponding author.

Authors' Contact Information: Jiadong Xie, The Chinese University of Hong Kong, China, jdxie@se.cuhk.edu.hk; Jeffrey Xu Yu, The Hong Kong University of Science and Technology (Guangzhou), China, jeffreyxuyu@hkust-gz.edu.cn; Siyi Teng, The Chinese University of Hong Kong, China, yu@se.cuhk.edu.hk; Yingfan Liu, Xidian University, China, liuyingfan@xidian.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2836-6573/2025/12-ART300
https://doi.org/10.1145/3769765

Table 1. Comparisons of Vector Search w/ and w/o Predicates

Approaches		QPS when recall@10=0.99			Index size (MB)	Index time (second)
		w/o pre.	cate.	nume.		
Single proximity graph	ADBV [40]	646.9	< 1	4.7	50.7	91.2
	Milvus [38]	646.9	< 1	6.1		
	VBASE [45]	646.9	1.1	10.4		
	ACORN [31]	559.5	< 1	< 1	202.8	3,229.3
		467.4	< 1	1.1	2,219.2	3,158.4
Multi-proximity graph	UNG [6]	459.5	57.6	-	263.9	856.8
	Serf [46]	453.7	-	13.0	441.8	1,097.7
	SuperPostfiltering [11]	508.8	-	23.2	5,492.6	2,470.1
	HSIG [22]	421.4	-	15.3	499.7	486.2
	iRange [42]	535.0	-	568.1	1,787.4	2,630.1
	FilteredDiskANN [15]	540.6	< 1	< 1	(cate.) 1,135.6 (nume.) 1,132.2	(cate.) 249.4 (nume.) 313.4
Our approach		646.9	187.9	542.6	(cate.) 178.1 (nume.) 130.4	(cate.) 41.9 (+91.2) (nume.) 32.5 (+91.2)

mixed-modality data [15, 28, 38, 40, 45], combining vectors with attributes. For instance, customers on an e-commerce platform might seek clothes resembling a reference image embedding (e.g., vector) while also specifying its product name, the color, the material, and price constraints, where the product name, color, and material are categorical attributes, and price is a numerical attribute [40, 45]. This necessitates exploring queries beyond the conventional k-ANN searches with additional predicates on attributes. Let $O = \{o_1, o_2, \cdots\}$ be a set of objects, where an object $o_i = (v_i, a_i)$ has a vector v_i and an attribute a_i, and an attribute a_i can be a set of categorical values or a number. A k-ANN query becomes $q = (v_q, c_q)$, where v_q is a user-given query vector and c_q is a predicate on the attribute. Note that $q = (v_q, c_q)$ is a conventional k-ANN search when $c_q = \emptyset$, $q = (v_q, c_q)$ can be used to search k-ANN with additional predicate on categorical/numerical attributes when $c_q \neq \emptyset$.

In the literature, numerous approaches have been proposed for addressing conventional k-ANN search [7, 9, 12, 13, 16, 18, 21, 24, 26, 32, 41, 43], i.e., $q = (v_q, \emptyset)$. Among them, proximity graph-based approaches have consistently demonstrated superior performance in various large-scale applications as validated in recent studies [4, 20, 21, 39]. However, they cannot directly be used to support k-ANN queries when $c_q \neq \emptyset$. We present it in Table 1, in which we compare the state-of-the-art approaches on Crawl dataset across various queries: when $c_q = \emptyset$ (denoted as "w/o pre."), when $c_q \neq \emptyset$ on categorical attributes (denoted as "cate.") and numerical attributes (denoted as "nume."), alongside their index size and index building time. We categorize Milvus [38], ADBV [40] and VBASE [45] as a single proximity graph approach. They achieve the best performance to support $q = (v_q, \emptyset)$. But, they cannot support queries efficiently when $c_q \neq \emptyset$. To support queries with predicates (i.e., $c_q \neq \emptyset$) efficiently, multi-proximity graph approaches are studied [6, 11, 15, 22, 42, 46]. They are designed to support a specific type of attributes, in other words, either categorical attributes [6] or numerical attributes [11, 22, 42, 46], but not both. Here, UNG [6] and iRange [42] are the state-of-the-art approaches for categorical and numerical attributes, respectively. Note that they cannot support conventional k-ANN efficiently, and their index size and index construction time are both much larger. There is an approach that can support all [15], but it cannot support any best.

In this paper, we study an all-in-one approach to support $q = (v_q, c_q)$ when $c_q = \emptyset$ and when c_q is on any of the two categorical/numerical attributes. As shown in Table 1, our approach achieves the best performance when $c_q = \emptyset$, outperforms UNG when the recall is 0.99 on categorical attributes, and achieves a similar performance to iRange on numerical attributes. Moreover, our index size and index construction time are small.

The main contributions of this work are summarized below. ❶ We support predicates without compromising the efficiency to support conventional k-ANN without predicates. In order to do

so, we take a single proximity graph G. In addition, we use an AND/OR graph $G_\sharp$ to handle the categorical/numerical attributes, where $G_\sharp$ is an AND-graph for categorical attributes and an OR-graph for numerical attributes. With G and $G_\sharp$, we construct an enhanced proximity graph G_+ with additional cross-edges that integrate G and $G_\sharp$. The number of nodes in $G_\sharp$ and the number of cross-edges in G_+ are controlled to be small. ❷ We design new algorithms to construct an AND/OR graph and an enhanced proximity graph. To construct an enhanced proximity graph, we study a cross-edges selection problem, which is proved to be NP-hard, and we propose an approximate algorithm with a theoretical guarantee. Also, we give a new search algorithm on the enhanced proximity graph to support k-ANN queries with and without predicates. ❸ We conduct extensive experimental results using real-world datasets. We confirm that our approach is effective and efficient. Our approach can be used to support any type of k-ANN queries without predicates or with predicates on categorical/numerical attributes.

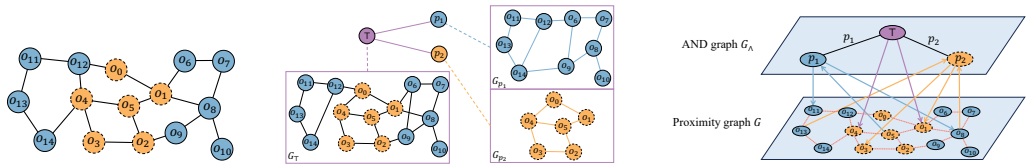
The paper is organized as follows. We give the preliminaries and the problem statement in Section 2, and discuss the limitations of the existing approaches for supporting vector search queries with and without predicates in Section 3. We highlight a new all-in-one approach in Section 4, discuss graph constructions in Section 5 and Section 6, and discuss searching in Section 7. We conduct extensive experimental studies to confirm the effectiveness and efficiency of our approach in Section 8. We conclude this work in Section 9.

2 Preliminaries and Problem Statement

Let $O = \{o_1, o_2, \cdots, o_n\} = \{(v_1, a_1), (v_2, a_2), \cdots, (v_n, a_n)\}$ be a dataset consisting of n objects, where each o_i is with a d-dimensional vector $v_i \in \mathbb{R}^d$, and an attribute a_i. The distance between two vectors, v_i and v_j, denoted as $\delta(v_i, v_j)$, is Euclidean distance in the L_2 norm. There are two types of attributes, namely, categorical attributes and numerical attributes. Let A_C be a domain of categorical values and A_N be a domain of numbers. An attribute a_i can be either a set of categorical values, $a_i = \{l_1, l_2, \cdots, l_{|a_i|}\}$, for $l_i \in A_C$, or a single value $a_i \in A_N$.

A vector search query (k-ANN query, or simply a query) is specified as $q = (v_q, c_q)$, where v_q is a user-given query vector in $\mathbb{R}^d$ and c_q is a predicate on attributes, which can be either a subset of categorical values, $c_q = \{l_1, l_2, \cdots\}$, or a range of numerical values $c_q = [q_l, q_r]$ for $q_l \leq q_r$, or empty $c_q = \emptyset$. When c_q is a subset of categorical values, an object $o_i = (v_i, a_i)$ satisfies the predicate if $c_q \subseteq a_i$. When c_q is a range of numerical values, an object $o_i = (v_i, a_i)$ satisfies the predicate if $a_i \in c_q$. When $c_q = \emptyset$, it implies that there is no condition and all objects satisfy the predicate. Let $O_{c_q} \subseteq O$ be the set of objects that satisfy c_q. The k-ANN query $q = (v_q, c_q)$ is to find the k-approximate nearest neighbors in O_{c_q} for the user-given query vector v_q. We use $Sel(c_q)$ to denote the fraction of objects that satisfy the predicate c_q such that $Sel(c_q) = |O_{c_q}|/|O|$. We use $recall@k$ to measure the quality of a k-ANN query q, which is defined as $|O_G \cap O_R|/k$, where O_G is the ground-truth object set which satisfies q, and O_R is the set found by a k-ANN algorithm for q.

For a k-ANN query $q = (v_q, \emptyset)$, the state-of-the-art approaches, (e.g., NSG [13] and HNSW [26]) can be used to support it using a proximity graph built. Here, a proximity graph is a directed graph $G = (V, E)$, where $v_i \in V$ represents a unique vector v_i (e.g., v_i in $o_i = (v_i, a_i) \in O$), and an edge, (v_i, v_j), in E represents that two vectors are close to each other based on the distance function $\delta(v_i, v_j)$, or precisely v_j is one of the k-nearest neighbors of v_i. In the existing proposed proximity graphs [13, 26, 39], the out-degree of each node, v_i, in G is consistently constrained within a predefined parameter M to facilitate efficient k-ANN search. Such a proximity graph cannot be used effectively to support a query $q = (v_q, c_q)$ when $c_q \neq \emptyset$, because all objects can be distributed over G and they are not connected. In other words, the k-ANN for a user-given query vector v_q is different over O_{c_q} and O. To deal with queries $q = (v_q, c_q)$ when $c_q \neq \emptyset$, there are approaches to



(a) A Single Proximity Graph (b) Multi-Proximity Graphs (c) Our Enhanced Proximity Graph

Fig. 1. Single/Multi Proximity Graphs

support categorical attributes [6, 15, 31], and numerical attributes [11, 22, 42, 46]. First, they cannot be efficiently used to support $q = (c_q, \emptyset)$. Second, the techniques used for categorical attributes cannot be effectively used to support numerical attributes, and vice versa. We will discuss their limitations next.

3 Limitations of Existing Approaches

In this section, we discuss the limitations of existing approaches to support a query $q = (v_q, c_q)$ for $c_q = \emptyset$ and $c_q \neq \emptyset$.

3.1 Single Proximity Graph Approaches

k-ANN search has been extensively studied using a proximity graph G built [13, 21, 26, 32, 39, 41]. Such approaches can support a query $q = (v_q, c_q)$ when $c_q = \emptyset$ efficiently. To further support a query $q = (v_q, c_q)$ for $c_q \neq \emptyset$, several studies study it by utilizing the single proximity graph built [31, 38, 40, 45]. Here, [38, 40, 45] focus on new search algorithms over the proximity graph built, and [31] focuses on building a modified proximity graph that supports predicates.

Search Algorithms: To support a query $q = (v_q, c_q)$ for $c_q \neq \emptyset$ over the proximity graph G built, Milvus [38] and ADBV [40, 44] study two k-ANN search strategies: (1) pre-filtering and (2) post-filtering. By pre-filtering, it first finds objects O_{c_q} that satisfy c_q and then searches over O_{c_q} for v_q. By post-filtering, it first performs k'-ANN search for $k' > k$ for v_q, and then searches the k-ANN satisfying the given predicate c_q over the k'-ANN found. As it is possible that there are no k-ANN satisfying c_q over the k'-ANN found, furthermore, VBASE [45] gives a new search algorithm based on relaxed monotonicity. Specifically, VBASE unfolds in two distinct phases to search by c_q and search by v_q. Initially, it disregards c_q, and searches towards vectors closer to v_q. Next, it considers c_q and explores the proximity graph G iteratively through the neighbors of the currently nodes that have been traversed.

<u>The Issues</u>: The pre-filtering works well if the selectivity of c_q, $Sel(c_q)$, is small. When $Sel(c_q)$ is large, it takes a long time to conduct an exhaustive search to find k-ANN over O_{c_q}. The post-filtering and the VBASE search algorithm inevitably traverse nodes within the proximity graph G that do not satisfy the predicate, otherwise, it may not be able to find k-ANN for a query. For example, a proximity graph G is shown in Fig. 1(a), where a node represents v_i and the color of a node in G represents a categorical value for an object $o_i = (v_i, a_i)$. Consider a query $q = (v_q, c_q)$. When $c_q = p_1$, the objects $\{o_6, \cdots, o_{14}\}$ satisfy p_1, and when $c_q = p_2$, the objects $\{o_0, \cdots, o_5\}$ satisfy p_2. It is easy to observe that when $q = (o_8, p_1)$ is to find k-ANN for $k = 6$, it cannot find all 6 objects as all objects $\{o_6, \cdots, o_{14}\}$ are separated in two disconnected subgraphs. In general, for a k-ANN query $q = (v_q, c_q)$, let O_{c_q} be the set of objects $\{o_i\}$ that satisfy c_q where $o_i = (v_i, a_i)$, and let V_p be the set of all vectors $\{v_i\}$ for O_p. The induced subgraph G_p over V_p in the proximity graph $G = (V, E)$ built is most likely to be disconnected, and the number of disconnected subgraphs cannot be bounded. In addition, let $V_{\neg p} = V \setminus V_p$, and the induced subgraph over $V_{\neg p}$ be $G_{\neg p}$. To navigate from a subgraph in G_p to another subgraph in G_p, it needs to traverse $G_{\neg p}$, and the number of nodes in $G_{\neg p}$ to

traverse cannot be bounded. The more nodes in $G_{\neg p}$ to be traversed, the higher overhead it occurs. Therefore, it is difficult to support a k-ANN query $q = (v_q, c_q)$ for $c_q \neq \emptyset$ using a single proximity graph G in practice regarding the accuracy and efficiency.

A 2-hop Proximity Graph: ACORN [31] builds a proximity graph $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ for a given proximity graph $G = (V, E)$ built [13, 16, 21, 26, 32, 39]. Here, for $\mathbf{G} = (\mathbf{V}, \mathbf{E})$, $\mathbf{V}$ is the same as V, a 2-hop neighbor of a node v_i (e.g., $(v_i, v_j)(v_j, v_k)$) in G is also a neighbor of v_i (e.g., (v_i, v_k)) in $\mathbf{G}$. ACORN [31] offers two variations for building $\mathbf{G}$: ACORN-γ and ACORN-1, which consider 2-hop neighbors during graph construction and search phase, respectively. Given the proximity graph $\mathbf{G}$, ACORN supports k-ANN query $q = (v_q, c_q)$ by traversing only nodes in $\mathbf{G}$ that satisfy the predicate c_q.
The Issues: First, comparing the proximity graph $G = (V, E)$ built, the proximity graph $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ is much larger with M^2 possibly additional edges (e.g., $|\mathbf{E}| = M^2|V|$). Recall M is the parameter used to control the number of out-going edges in a proximity graph G. It is important to note that additional edges in $\mathbf{G}$ have a great impact on the performance of k-ANN search, as more edges need to be traversed. ACORN proposes strategies to compress $\mathbf{G}$. But, the memory consumption remains high, as shown in Exp. 2 and 5 in our performance studies. Second, ACORN assumes that the induced subgraph G_P over the vectors that satisfy c_q in $\mathbf{G}$ is connected, which is unlikely to be true always. For example, in Fig. 1(a), the sets $\{o_6, \cdots, o_{10}\}$ and $\{o_{11}, \cdots, o_{14}\}$ are not connected even if a 2-hop neighbor of a node, v_i is taken as a neighbor of the node v_i.

3.2 Multi-proximity Graph Approaches

To deal with k-ANN query $q = (v_q, c_q)$ for $c_q \neq \emptyset$, multi-proximity graph approaches are proposed to support predicates (e.g., c_q) using specific proximity graphs. The ideal case is to have a proximity graph G_{c_q} for a specific predicate c_q. As the number of distinct categorical/numerical values can be large, a key issue is how to select attribute values, $P = \{p_1, p_2, \cdots\}$, to build a specific proximity graph G_{p_i} for every p_i, and support any predicate using a combination of proximity graphs built.

Reconsider Fig. 1(a). It shows a set of objects, O, where every object, o_i has one of two categorical values represented by two different colors, denoted as p_1 and p_2. Let O_{p_1} and O_{p_2} be the objects in O that have categorical values, p_1 and p_2, respectively. As shown in Fig. 1(b), a multi-proximity graph approach may build four proximity graphs, G_{p_1} for O_{p_1}, G_{p_2} for O_{p_2}. Note that, $G_\top$ for objects that have any of the two categorical values, which is different to the proximity graph built for Fig. 1(a). It is because, different to the construction of the proximity graph in Fig. 1(a), the graph $G_\top$ is constructed by adding the nodes from graph G_{p_2} into G_{p_1}. For example, o_{14} is not the top-3 nearest nodes of either o_4 or o_3, resulting in the edge from either o_3 or o_4 to o_{14} being absent.

There are approaches to deal with categorical attributes [6, 15] and numerical attributes [11, 22, 42, 46]. Below, we exemplify two state-of-the-art approaches, UNG [6] for categorical values and iRange [42] for numerical values.
UNG [6]: Recall A_C is the domain of categorical values, and a_i for an object $o_i = (v_i, a_i)$ is a subset of categorical values. For a set of n objects, O, let $F = \bigcup_{i=1}^{n}\{a_i\}$ be the set of all distinct subsets of categorical values. UNG builds a proximity graph G_f for every $f \in F$, which contains all objects, $o_i = (v_i, a_i)$ for $a_i = f$. There are in total $|F|$ proximity graphs, and each object is maintained in at most one proximity graph. The total size for all proximity graphs built is $O(M \cdot |O|)$ where M is the maximum number of out-going degrees. UNG also maintains a navigating graph $G_N = (V_N, E_N)$, where $V_N = F$, and there is an edge from f to f' if f' is the minimum superset of f among all sets in F. It is worth noting that the predicate c_q in a query $q = (v_q, c_q)$ can be any subset of A_C which does not necessarily appear in F. For a given arbitrary query predicate, c_q, UNG needs to identify the nodes in G_N, which are the minimal supersets of c_q, and search the corresponding proximity graphs built for those nodes in G_N. As an example, suppose F contains all subsets of categorical

Table 2. Sizes of Index and Minimal Coverage Set

Methods	Index size		Size of MCS	
	Categorical	Numerical	Categorical	Numerical
iRange [42]	-	$O(M \cdot \lvert O \rvert \cdot \log \lvert A_N \rvert)$	-	$O(\log \lvert A_N \rvert)$
UNG [6]	$O(M \cdot \lvert O \rvert)$	-	$O(\min(2^{\lvert A_C \rvert}, \lvert O \rvert))$	-
Ours	$O(\min(M \cdot \lvert O \rvert, M_p))$	$O(\min(M \cdot \lvert O \rvert, M_p))$	$O(1)$	$O(1)$

values in the same size x ($x \leq |A_C|$). When c_q contains a single categorical value, it needs to search $\binom{|A_C|-1}{x-1}$ proximity graphs. UNG [6] may need to retrieve $|F|$ proximity graphs, which are bounded by $O(\min(2^{|A_C|}, |O|))$.

iRange [42]: It builds a classical segment tree [8] for numerical attribute values, where a node in the tree represents a continuous range, $[l, r]$. Accordingly, for a tree node, a proximity graph is built for objects, $o_i = (v_i, a_i)$, whose numerical value, a_i is in the range. The segment tree contains $O(|A_N| \log |A_N|)$ nodes, and each object in O is maintained by $O(\log |A_N|)$ tree nodes or $O(\log |A_N|)$ proximity graphs. This results in a total index size of $O(M \cdot |O| \cdot \log |A_N|)$. Note that, for numerical values, c_q in a query $q = (v_q, c_q)$, is a range such that $c_q = (q_l, q_r)$, iRange needs to identify the tree nodes in the segment tree that cover c_q and search the corresponding proximity graphs, which is bounded by $O(\log |A_N|)$.

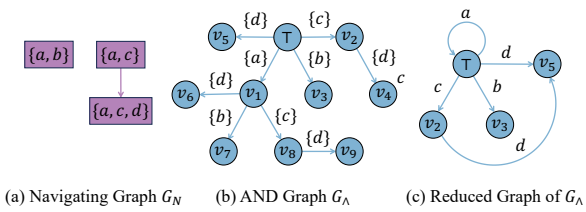
We summarize the index sizes and size of MCS in Table 2, in which MCS refers to the number of minimal coverage sets (MCS) or the number of nodes in either the navigating graph or the segment tree that need to be searched.

Issues: First, all multi-proximity graph approaches are designed to support a query $q = (v_q, c_q)$ for $c_q \neq \emptyset$, their search performance may not match that of state-of-the-art approaches for queries without predicates. It is because they either do not support all types of proximity graphs, e.g., iRange and Serf only support NSW [25] or HNSW [26] in their index; or compromise the quality of the proximity graph, thus can not support a query when $c_q = \emptyset$ efficiently. For example, given a proximity graph $G_\top$ in Fig. 1(b), consider a query $q = (v_{14}, \emptyset)$, where v_{14} is the vector for the object o_{14}. Suppose that the search starts from the vector v_5 for o_5 on the proximity graph $G_\top$ illustrated in Fig. 1(b), it cannot navigate to v_{14} and find o_{14}. We further confirm this limitation in our Exp. 3. Second, despite the fact that it builds many proximity graphs to support attribute values, it needs to search many proximity graphs or search on the merged proximity graphs to answer a query with predicates. The efficiency depends on the predicate (e.g., c_q). Third, the index size and index construction time are large. UNG [6] builds $|F|$ proximity graphs, whereas iRange [42] builds $O(|A_N| \log |A_N|)$ proximity graphs. For each proximity graph, it needs to be built from scratch, the total index building time is large.

4 A New All-in-one Approach

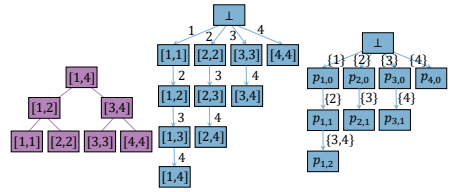
None of the existing approaches can support a k-ANN query $q = (v_q, c_q)$ when $c_q = \emptyset$ and $c_q \neq \emptyset$, and support both categorical attributes and numerical attributes effectively and efficiently. For a system to support all, it needs to build a proximity graph G to support the case when $c_q = \emptyset$. To support c_q for categorical attributes, it needs to build additional $O(\min(2^{|A_C|}, |O|))$ proximity graphs, and to support c_q for numerical attributes, it needs to build additional $O(|A_N| \log |A_N|)$ proximity graphs. It is impractical to support any k-ANN query $q = (v_q, c_q)$ in such a way.

In this section, we propose a new approach to support all. The key idea is to make use of the proximity graph G built for $c_q = \emptyset$ to support $c_q \neq \emptyset$ as well. In order to do so, we build an AND/OR graph, denoted $G_\wedge/G_\vee$, for objects in O with categorical/numerical attributes. It is important to mention that we do not build any additional proximity graphs for a node in an AND/OR graph. Instead, we enhance the proximity graph G, denote it as G_+, by adding a limited



(a) Navigating Graph G_N (b) AND Graph $G_\wedge$ (c) Reduced Graph of $G_\wedge$

Fig. 2. An AND Graph $G_\wedge$



(a) Segment Tree (b) OR Graph $G_\vee$ (c) Reduced Graph of $G_\vee$

Fig. 3. An OR Graph $G_\vee$

number of cross-edges in both directions from/to the AND/OR graph. Fig. 1(c) shows an enhanced G_+ with a proximity graph G and an AND graph $G_\wedge$.

Below, we discuss the AND/OR graph with examples in detail.

AND/OR Graph: An AND/OR graph is a rooted DAG (Directed Acyclic Graph) with edge labels, where we denote an AND graph as $G_\wedge = (V, E)$, and an OR graph as $G_\vee = (V, E)$. In the following, we use $V(G)$ and $E(G)$ to denote the set of nodes and the set of edges for a graph G, respectively. A node u, in an AND/OR graph, has an extent, denoted as $\text{ext}(u)$, which represents a subset of objects in $\mathcal{O}$. An AND graph $G_\wedge$ is for objects $\mathcal{O}$ with categorical values, $A_C = \{l_1, l_2, \cdots\}$. For the root, denoted as $\top$, its $\text{ext}(\top)$ is the entire set of objects $\mathcal{O}$. Let u and v be two nodes in $G_\wedge$, and there is an edge from u to v with an edge-label $L_{u,v}$ ($\subset A_C$). Suppose $\text{ext}(u) = \mathcal{O}_u$ and $\text{ext}(v) = \mathcal{O}_v$, which are subsets of $\mathcal{O}$. With the edge, it specifies that $\text{ext}(v)$ is a subset of $\text{ext}(u)$ such that an object in $\text{ext}(v)$ is an object in $\text{ext}(u)$ that also has the categorical attributes in $L_{u,v}$, i.e., $\text{ext}(v) = \text{ext}(u) \cap \mathcal{O}_{L_{u,v}}$. And the extent of a node u, $\text{ext}(u)$, represents the objects that satisfy all the edge-labels associated in a conjunctive way along the path from the root to itself, denoted as $\text{pred}(u)$. In a similar manner, an OR graph $G_\vee$ is for objects $\mathcal{O}$ with numerical values, A_N. Here, the root, denoted as $\perp$, has an extent $\text{ext}(\perp) = \emptyset$. Let u and v be two nodes in $G_\vee$, and there is an edge from u to v with an edge-label $L_{u,v}$ ($\in A_N$). Suppose $\text{ext}(u) = \mathcal{O}_u$ and $\text{ext}(v) = \mathcal{O}_v$, which are a subset of $\mathcal{O}$. With the edge, it specifies that $\text{ext}(v)$ is a superset of $\text{ext}(u)$ with additional objects that have the label $L_{u,v}$, $\text{ext}(v) = \text{ext}(u) \cup \mathcal{O}_{L_{u,v}}$. A restriction imposed on the OR graph, $G_\vee$, for numerical values is that the edge labels must be continuous. That is, let u, v, and w be three nodes with two edges (u, v) and (v, w). If the edge-label of (u, v) is a number a_i, the edge-label of (v, w) must be a_j, where a_i and a_j are two continuous numerical values in A_N, i.e., $\nexists a_k \in A_n, a_i < a_k < a_j$. And the extent of a node u, $\text{ext}(u)$, represents the objects that satisfy all the edge-labels associated in a disjunctive way along the path from the root to itself, denoted as $\text{pred}(u)$. For a query predicate c_q, we aim to find a node set $\{p_1, \cdots, p_m\}$ in an AND/OR graph with minimal size m such that $\mathcal{O}_{c_q} = \bigcup_{i=1}^{m} \text{ext}(p_i)$, where $p_i \in V(G)$. We refer to this minimal set for c_q as the minimal coverage set (MCS) of c_q.

Example 4.1: Consider $\mathcal{O} = \{o_1, o_2, o_3\}$ where $A_C = \{a, b, c, d\}$. Suppose o_1, o_2, and o_3 have a subset of categorical values, $\{a, b\}$, $\{a, c\}$, and $\{a, c, d\}$, respectively. Fig. 2(a) shows the navigating graph G_N used by UNG [6]. Fig. 2(b) shows an AND graph $G_\wedge$. Here, the root $\top$ represents $\mathcal{O}$, v_1 represents $\mathcal{O}$ because all the three objects have an attribute value a, and v_8 represents $\{o_2, o_3\}$ because these two objects have a subset of attribute values $\{a, c\}$. Moreover, $\text{pred}(v_8) = \{a, c\}$, since there is a path from $\top$ to v_8 with two edges labeled $\{a\}$ and $\{c\}$. Fig. 2(c) shows a reduced $G_\wedge$, where the nodes that represent the same subset of objects are merged. For example, the root $\top$ and v_1 are merged to be $\top$ as they both represent $\mathcal{O}$, v_2 and v_8 are merged to be v_2, nodes v_3 and v_7 merged to be v_3, and nodes $\{v_4, v_5, v_6, v_9\}$ are merged to be v_5.

Example 4.2: Consider $\mathcal{O} = \{o_1, o_2, o_3, o_4\}$, where each o_i is associated with a numerical value i, i.e., o_1, o_2, o_3, o_4 have numerical attributes $1, 2, 3, 4$ respectively. The segment tree used in iRange [42] is

shown in Fig. 3(a). Fig. 3(b) shows an OR graph $G_\vee$. Here, the root $\perp$ represents an empty set of O. The root node $\perp$ has four children, where each represents a subset of objects in a range $[i, i]$, that contains additional objects with the ones represented by the root. For example, $[1, 1]$ is for $\{o_1\}$. The child node $[1, 2]$ is for $\{o_1, o_2\}$. Fig. 3(c) illustrates the notation we will use to construct a reduced graph $G_\vee$. Here, a node $p_{i,j}$ represents a range $[i, i + 2^j - 1]$. For example the node $p_{1,2}$ represents a range $[1, 4]$, which represents the objects that have a numerical value in that range.

The AND/OR Graph Size: To control the size of an AND/OR graph, $G_\wedge/G_\vee$, we use a parameter M_p to ensure that its number of nodes is less than or equal to M_p. We will discuss how to reduce the set of nodes in an AND/OR graph to be at most M_p later. Some simple strategies can be used to reduce the number of nodes. ❶ (Merging Nodes with Identical Extent) We merge nodes with identical extents. That is, if nodes u and v possess the same subset of objects, i.e., $\text{ext}(u) = \text{ext}(v)$, we merge nodes u and v into a single node representing $\text{ext}(u) \cup \text{ext}(v)$. ❷ (Merging Nodes with Small Selectivities) We merge nodes whose extents are small or with small selectivities ($Sel(\cdot)$), and conduct the pre-filtering method to answer the vector search queries for these nodes. It is because, following the experiments conducted in [38, 40], pre-filtering has the best performance when the selectivity of the query is small.

An Enhanced Proximity Graph G_+: Let $G = (V, E)$ be a proximity graph, and $G_\sharp = (V_\sharp, E_\sharp)$ be an AND/OR graph, which can be either $G_\wedge$ or $G_\vee$. An enhanced proximity graph $G_+ = (V_+, E_+)$ is a directed graph, where $V_+ = V \cup V_\sharp$ and $(E \cup E_\sharp) \subset E_+$. The edges in $E_+ \setminus (E \cup E_\sharp)$ are called cross-edges which can be from V to $V_\sharp$ or from $V_\sharp$ to V. We call the former P♯-edges and call the latter ♯P-edges. Here, an ♯P-edge, (u, v) for $u \in G_\sharp$ and $v \in G$, connects u to v in a way that v is a node that satisfies the predicates specified by $\text{pred}(u)$. Such ♯P-edges can help to find nodes in the proximity graph that satisfy the predicate c_q in an k-ANN query $q = (v_q, c_q)$. A P♯-edge, (w, u) for $u \in G_\sharp$ and $w \in G$, connects w in a proximity graph G to a node $v \in G$ in a proximity graph G via a node in $G_\sharp$. Since it needs to maintain G in its best form to support queries $q = (v_q, \emptyset)$, we do not modify G itself; instead, we add special edges from G to $G_\sharp$. Those special edges (i.e., P♯-edges and P♯-edges) can be ignored when supporting query $q = (v_q, \emptyset)$. For queries with predicates, at a high level, P♯-edges with P♯-edges are used to add edges from one node to another node in the proximity graph G for search. We explain it by an example.

Example 4.3: Reconsider Fig. 1(a), its enhanced proximity graph G_+ is shown in Fig. 1(c) with an AND graph $G_\wedge$ at the top and a proximity graph G at the bottom. There are ♯P-edges (p_1, o_8), (p_1, o_{11}). With such edges, two disconnected subgraphs that have the same attribute p_1 are connected with node p_1. There are P♯-edges, for example, (o_5, p_1). We explain its usage. Consider a query $q = (v_q, p_1)$. During searching over the proximity graph G, it may encounter a node (e.g., o_5) which does not have p_1 as its attribute, with such an edge (o_5, p_1), it can bring the search to objects that have p_1 as its attribute through node p_1 and ♯P-edges from p_1.

We discuss cross-edges in detail. The number of cross-edges needs to be controlled and the cross-edges selection must be optimized. A set of objects O be divided into two disjoint subsets, O_p and $O_{\neg p}$, by a predicate p, where O_p is the set of objects that satisfy the predicate p and $O_{\neg p}$ does not. Below, we use V_p and $V_{\neg p}$ to refer to the set of vectors for O_p and $O_{\neg p}$. The ♯P-edges are selected to minimize the maximum hop distance from a node in V_p to any node in V_p. And the P♯-edges are selected to minimize the maximum hop distance from a node in $V_{\neg p}$ to any node in V_p.

Definition 4.1: (**Cross-edges Selection Problem**) Given a graph G, a node set S in G, and an integer M. Let $G[S]$ be the induced subgraph of G over the set of nodes S. The cross-edges selection problem is to add M edges from a new node $v \notin G$ to M nodes in S, such that the maximum hop distance in G from v to any node in S on $G[S \cup \{v\}]$ is minimized.

Algorithm 1: ConstructAndG ($\mathcal{O}, A_C, M_p$)

Input : a dataset $\mathcal{O}$, a categorical attribute domain A_C, M_p to limit the number of nodes in $G_\wedge$
Output : An AND graph $G_\wedge$

1 Add the root $\top$ to $G_\wedge$;
2 $Q \leftarrow \top$; ext[$\top$] $\leftarrow \mathcal{O}$; map[$\mathcal{O}$] $\leftarrow \top$;
3 **for** each $l \in A_C$ in order of $Sel(l_1) \geq \cdots \geq Sel(l_{|L|})$ **do**
4 **for** each node u in Q **do**
5 $\mathbb{E} \leftarrow$ ext(u) $\cap V_l$ where V_l is the objects whose label has l;
6 **if** $\mathbb{E} = \emptyset$ **then continue**;
7 **if** map[$\mathbb{E}$] $\neq \emptyset$ **then**
8 Add an edge from u to map[$\mathbb{E}$] labeled l to $G_\wedge$;
9 **else**
10 **if** $|V(G_\wedge)| = M_p$ **then**
11 Add an edge (u, u) labeled $\neg l$ to $G_\wedge$; **continue**;
12 Add a new node v and an edge (u, v) labeled l to $G_\wedge$;
13 map[$\mathbb{E}$] $\leftarrow v$; ext(v) $\leftarrow \mathbb{E}$;
14 $Q \leftarrow V(G_\wedge)$;
15 **return** $G_\wedge$

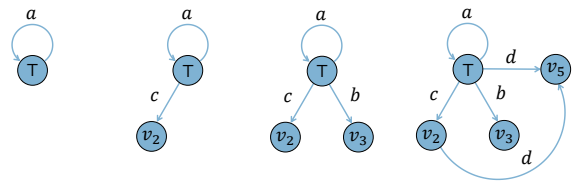
For ♯P-edges selection, G is the proximity graph, S is V_P that satisfies a predicate p, and v is the node in $G_\sharp$ with the predicate p. For P♯-edges, since the proximity graph is fixed, we consider reducing the maximum hop distance from $V_{\neg p}$ to V_p by reducing the maximum distance from $V_{\neg p}$ to V_p through the nodes p in the AND/OR graph. As we have minimized the hop distance from p to V_p by introducing the ♯P-edges, the problem becomes adding P♯-edges to minimize the maximum hop distance from each node in $V_{\neg p}$ to p. Considering the problem on the reverse graph of the proximity graph, we can find that it transforms into minimizing the maximum hop distance from p to $V_{\neg p}$. Hence, for the P♯-edges selection, in the cross-edges selection problem, G is the reverse graph of the proximity graph, S is $V_{\neg p}$, and v is the node in $G_\sharp$ with the predicate p. Note that, although the selection of ♯P-edges and P♯-edges can be modeled into the same cross-edge selection problem, they are two independent problems with distinct objectives. We will discuss the algorithm for them in Section 6.

5 Graph Construction

In this section, we discuss how to construct AND/OR graphs for objects with categorical/numerical attributes, and how to construct an enhanced proximity graph.

5.1 AND Graph Construction

We present a ConstructAndG algorithm in Algorithm 1 to construct an AND graph $G_\wedge$. Initially, we add a root node, $\top$, to $G_\wedge$ (line 1). The extent of $\top$ is the entire set of objects $\mathcal{O}$. We use map[O'] to keep the nodes in $G_\wedge$ that extent is O' (line 2). Next, we construct $G_\wedge$ in descending order of label selectivity (lines 3-14). It is because we merge nodes that represent labels with lower selectivity when the number of graph nodes reaches the specified upper limit M_p at lines 10-11. For each existing node u in $G_\wedge$ (line 4), we establish an edge from u to a node v with the new label l if there exists at least one object satisfying both l and the predicates pred(u) (lines 5-6). Nodes with the same subset of objects are merged (lines 7-8). If the set of objects satisfying both l and the predicate represented by u is absent in $G_\wedge$, then we add a new node, v, to $G_\wedge$ if $|V(G_\wedge)| < M_p$ (lines 12-13).



(1) Add label a (2) Add label c (3) Add label b (4) Add label d

Fig. 4. An Example of Constructing An AND Graph

Algorithm 2: SearchAndG ($G_\wedge, c_q$)

Input : an AND graph $G_\wedge$ and a query predicate c_q
Output : a single node set $\{p\}$ in $G_\wedge$ to search

1 $p \leftarrow \top$, the root of $G_\wedge$;
2 **for** each $l_i \in c_q$ in order of $Sel(l_1) \geq Sel(l_2) \geq \cdots$ **do**
3 **if** there is an edge from p to p' with labeled l_i or $\neg l_i$ **then**
4 $p \leftarrow p'$;
5 **else return** $\emptyset$;
6 **return** $\{p\}$;

Otherwise, we add an edge (u, u) to denote the merging (lines 10-11), i.e., merging this node with u. The set of nodes in $G_\wedge$ is pushed to Q (line 14).

Example 5.1: Let $O = \{o_1, o_2, o_3\}$, where o_1, o_2, and o_3 have categorical attributes $\{a, b\}$, $\{a, c\}$, and $\{a, c, d\}$, respectively. Thus, $Sel(\{a\}) > Sel(\{c\}) > Sel(\{b\}) = Sel(\{d\})$. Fig. 4 shows the process of constructing $G_\wedge$ following the order of (a, c, b, d). Initially, we add a node v_1, and an edge from $\top$ to v_1 with label a. However, upon finding $\text{ext}(v_1) = \{o_1, o_2, o_3\} = \text{ext}(\top)$, we decide to merge v_1 with $\top$, and we remove v_1 and its edge, and add an edge $(\top, \top)$ labeled a. Next, for label c, we create a node v_2, and linked $\top$ to v_2 labeled c, where $\text{ext}(v_2) = \{o_2, o_3\}$ and $\text{map}[\{o_2, o_3\}] = v_2$. Continuing, for label b, we add two nodes, v_3 and v_4, and add edges labeled b from $\top$ to v_3 and from v_2 to v_4 respectively. Here, $\text{ext}(v_3) = \text{ext}(\top) \cap V_{\{b\}} = \{o_1\}$, and $\text{map}[\{o_1\}] = v_3$; however, as $\text{ext}(v_4) = \text{ext}(v_2) \cap V_{\{b\}} = \emptyset$, v_4 and the edge (v_2, v_4) are removed. Finally, for label d, nodes v_5, v_6, and v_7 are added and linked with edges labeled d from $\top$ to v_5, v_2 to v_6, and v_3 to v_7, respectively. Here, $\text{ext}(v_5) = \text{ext}(root) \cap V_{\{d\}} = \{o_3\}$, and $\text{map}[\{o_3\}] = v_5$. Then, $\text{ext}[v_6] = \text{ext}[v_2] \cap V_{\{d\}} = \{e_3\}$, hence, v_6 is merged with v_5, i.e., removing v_6 and adding an edge from v_2 to v_5 labeled d. Lastly, as $\text{ext}(v_7) = \text{ext}(v_3) \cap V_{\{d\}} = \emptyset$, v_7 is eliminated.

Given $G_\wedge$ constructed, the search algorithm to find the nodes in $G_\wedge$ to start searching is given in SearchAndG algorithm (Algorithm 2). The navigation on $G_\wedge$ starts from the root $\top$ (line 1). For each label in the query predicate, we traverse it in descending order of selectivity, aligning the construction order of AND graph. Here, the selectivity refers to the selectivity of labels within the datasets, which can be preprocessed during the index construction phase. At each time, we traverse to a node p' linked from the current node p with the corresponding label (lines 3-4). The search concludes if no such node p' exists, indicating an empty set of objects to be found for the given query predicate, c_q (line 5). It is important to note that during navigation, if it encounters an edge labeled with $\neg l$, it implies that the node representing the predicate is merged. In such a case, we opt for pre-filtering to obtain results instead of employing our search algorithm (Algorithm 7).

Unlike UNG, which needs a rather large number of nodes to start searching, bounded by $O(\min(2^{|A_C|}, |O|))$ and also needs $O(\min(2^{|A_C|}, |O|))$ time to find them, using the navigating

Algorithm 3: ConstructOrG (O, A_N, M_p)

Input : a dataset O, a numerical attribute domain A_N, M_p to limit the number of nodes in $G_\vee$
Output : An OR graph $G_\vee$

1 **for** each $a_i \in A_N$ in order of $a_1 < a_2 < \cdots < a_{|A_N|}$ **do**
2 $\quad l \leftarrow \lfloor \log(|A_N| - i + 1) \rfloor$;
3 $\quad$ **for** $j \leftarrow 0$ **to** l **do**
4 $\quad\quad$ Add a node $p_{i,j}$ and an edge $(p_{i,j-1}, p_{i,j})$ with label $\{a_{i+2^{j-1}}, \cdots, a_{i+2^j-1}\}$ to $G_\vee$ where $p_{i,-1}$ is the root $\perp$;
5 $\quad\quad$ Set the range indicated by $p_{i,j}$ as $[a_i, a_{i+2^j-1}]$;
6 **while** $|V(G_\vee)| > M_p$ **do**
7 $\quad$ Find two nodes, $p_{i,j}$ and $p_{i,k}$, in $G_\vee$ such that $0 \le j < k$ and the number of entities in O satisfying the range of $p_{i,j}$ is minimized;
8 $\quad$ Add an edge from $p_{i,j}$'s parent to $p_{i,k}$ in $G_\vee$;
9 $\quad$ Remove the node $p_{i,j}$ and its edges in $G_\vee$;
10 **return** $G_\vee$

graph. The node that we need to start searching is one, and finding it needs $O(|c_q|)$ time, as can be observed in Algorithm 2, where $|c_q|$ is the number of labels included in the query predicate.

5.2 OR Graph Construction

Without loss of generality, we assume the numerical attributes $a_1, a_2, \cdots, a_{|A_N|}$ in A_N satisfying $a_1 < a_2 < \cdots < a_{|A_N|}$. In an OR graph $G_\vee$, we use $O(\log(|A_N| - i + 1))$ nodes to represent the range for each $a_i \in A_N$, with the left endpoint in the range being a_i. Specifically, for each $a_i \in A_N$, let l be the maximum integer such that $2^l \le |A_N| - i + 1$; we utilize l nodes to depict the range starting from i. For $a_i \in A_N$ and $j \in \{1, 2, \cdots, l\}$, where $2^l \le |A_N| - i + 1$, we denote the j-th node of a_i in $G_\vee$ as $p_{i,j}$, where $p_{i,j}$ represents the range from a_i to a_{i+2^j-1}, and we use $\text{range}(p_{i,j})$ to denote the range of node $p_{i,j}$ such that $\text{range}(p_{i,j}) = [a_i, a_{i+2^j-1}]$.

The construction of an OR graph, $G_\vee$ is given in Algorithm 3. For every $a_i \in A_N$ (line 1), we determine the number of nodes l to depict the range commencing at a_i (line 2). Next, such nodes and edges are added to $G_\vee$ (lines 3-5). We construct $G_\vee$ under the constraint that $|V(G_\vee)| \le M_p$ for a given parameter M_p. When $G_\vee$ has more nodes, we merge nodes using a while loop (lines 6-9). Here, each pair of nodes in $G_\vee$ has different extents, i.e., different sets of objects. The sole approach to restricting the $|V(G_\vee)|$ is by consolidating nodes with lower selectivity. Thus, at line 7, we identify a node with the smallest selectivity in $G_\vee$, and consider its removal if another node denotes the same range starting points (lines 7-9). Note that the value of M_p must be set greater than $|A_N|$.

Example 5.2: Consider $A_N = \{1, 2, 3, 4\}$ in Example 4.2, we first add nodes for $a_1 = 1$. With $l = \lfloor \log 4 \rfloor = 2$, as depicted in Fig. 3(c), nodes $p_{1,0}$, $p_{1,1}$, and $p_{1,2}$ are added along with edges $(\perp, p_{1,0})$, $(p_{1,0}, p_{1,1})$, and $(p_{1,1}, p_{1,2})$ labeled with $\{1\}$, $\{2\}$, and $\{3, 4\}$ respectively. Next, for $a_2 = 2$ and $a_3 = 3$, where $l = 1$, we add nodes $p_{2,0}$, $p_{2,1}$, $p_{3,0}$, $p_{3,1}$ with the edges $(\perp, p_{2,0})$, $(p_{2,0}, p_{2,1})$, $(\perp, p_{3,0})$, and $(p_{3,0}, p_{3,1})$, labeled $\{2\}$, $\{3\}$, $\{3\}$, $\{4\}$ respectively, in sequence. For $a_4 = 4$, we have $l = 1$, we add $p_{4,0}$ with edge $(\perp, p_{4,0})$ labeled $\{4\}$.

Theorem 5.1: *Given an OR graph $G_\vee$ with numerical attributes $A_N = \{a_1, a_2, \cdots, a_{|A_N|}\}$ for $a_1 < a_2 < \cdots < a_{|A_N|}$. For an arbitrary query predicate $c_q = [a_l, a_r]$, where $1 \le l \le r \le |A_N|$, we can find two nodes, $p_{l,m}$ and $p_{r-2^m+1,m}$, in $G_\vee$ such that $[a_l, a_r] = \text{range}(p_{l,m}) \cup \text{range}(p_{r-2^m+1,m})$, where $m = \lfloor \log(r - l + 1) \rfloor$.*

Algorithm 4: SearchOrG ($G_\vee$, (q_l, q_r))

Input : an OR graph $G_\vee$ and a numerical range (q_l, q_r)
Output : the set of nodes $\{p_1, \cdots, p_m\}$ in $G_\vee$ to search

$l \leftarrow$ the minimum value of i such that $a_i \geq q_l$;
$r \leftarrow$ the maximum value of i such that $a_i \leq q_r$;
$m \leftarrow \lfloor \log(r - l + 1) \rfloor$;
if $2^m = r - l + 1$ **then return** $\{p_{l,m}\}$;
else return $\{p_{l,m}, p_{r-2^m+1,m}\}$;

Algorithm 5: ConstructEnhancedG ($\mathcal{O}, G, A, M_p, M$)

Input : a dataset $\mathcal{O}$, a proximity graph G, an attribute set A, M_p to limit the number of nodes in an AND/OR graph, and M to limit the number of cross-edges
Output : the enhanced proximity graph G_+

1 Construct an AND/OR graph $G_\sharp$ by either Algorithm 1 or 3;
2 $G_+ \leftarrow G \cup G_\sharp$;
3 **for** each node p in $G_\sharp$ **do**
4 Let V_p be the nodes in G that satisfy p;
5 Let $V_{\neg p}$ be the nodes in G that do not satisfy p;
6 $V_1 \leftarrow$SelectCrossEdge (G, V_p, M); // Section 6
7 Let $G_\downarrow$ be the graph by reversing the edge directions in G;
8 $V_2 \leftarrow$SelectCrossEdge ($G_\downarrow, V_{\neg p}, M$); // Section 6
9 $CE \leftarrow \{(p, v) \mid v \in V_1\} \cup \{(v, p) \mid v \in V_2\}$;
10 Add edges in CE to G_+;
11 **return** G_+

Proof Sketch: In OR graph construction, the nodes $p_{l,m}$ and $p_{r-2^m+1,m}$ represent the ranges $[l, l + 2^m - 1]$ and $[r - 2^m + 1, r]$, respectively. Given that $m = \lfloor \log(r - l + 1) \rfloor$, it follows that $l+2^m-1 \leq l+(r-l+1)-1 = r$ and $r-2^m+1 \geq r-(r-l+1)+1 = l$. In addition, $2^{m+1} > 2 \cdot \left\lfloor \frac{r-l+1}{2} \right\rfloor \Leftrightarrow 2^m + 2^m \geq r - l + 2 \Leftrightarrow l + 2^m - 1 \geq r - 2^m + 1$. Hence, $[a_l, a_r] = \text{range}(p_{l,m}) \cup \text{range}(p_{r-2^m+1,m})$. □

Based on Theorem 5.1, we design Algorithm 4 to identify the nodes in $G_\vee$ to initiate the search. It is worth noting that the nodes returned Algorithm 4 may not exist in $G_\vee$ due to merging strategies, where the merged nodes have lower selectivity. Thus, for such returned nodes, we employ pre-filtering to acquire the results instead of utilizing our search algorithm (Algorithm 7).

The number of nodes in $G_\vee$ is $O(\min(M_p, |A_N| \log |A_N|))$, which is less than or equal to the number of nodes in the segment tree utilized in iRange [42], i.e., $O(|A_N| \log |A_N|)$. In contrast, our method requires a significantly smaller number of nodes to start searching for a predicate c_q and needs less time to find these nodes. That is, we only require $O(1)$ time for Algorithm 4 to identify at most 2 nodes to cover the query predicate c_q, whereas iRange needs $O(\log |A_N|)$ time to find a node set of size at most $O(\log |A_N|)$.

5.3 Enhanced Proximity Graph

We present ConstructEnhancedG algorithm (Algorithm 5) to construct the enhanced proximity graph G_+. Initially, we create an AND/OR graph, $G_\sharp$, based on the categorical/numerical attributes (line 1). Next, the nodes in our graph index encompass those from the given proximity graph G and the constructed AND/OR graph $G_\sharp$ (line 2). For each node p in $G_\sharp$, that represents a predicate p (line 3), we identify the set of nodes, V_p, in the proximity graph, G, where the corresponding

objects satisfy the predicate p (line 4), and the set of nodes, $V_{\neg p}$, where the corresponding objects do not satisfy the predicate p (line 5). We select $\sharp$P-edges and P$\sharp$-edges by utilizing these two sets (lines 6-8). Such cross-edges are merged and are included in G_+ (lines 9-10). The details of the algorithm for the cross-edge selection problem will be discussed in Section 6.

Dealing with Data Update: To support data updates, three components need to be considered for updating: the proximity graph G, the AND/OR graph $G_\sharp$, and cross-edges. For the proximity graph G, we can employ state-of-the-art methods [36, 38] to rebuild the proximity graph once the changes (insertions and deletions) accumulate to a specific threshold. To update AND/OR graph, we do it as follows. First, attributes insertion can be handled because AND/OR graph is constructed through incremental insertion. Second, for attributes deletion, we mark it on nodes without real deletion. Third, for cross-edges, we only need to deal with insertion when a new node is introduced to AND/OR graph, i.e., by calling Algorithm 6 directly. We will reconstruct AND/OR graph and the cross edges when rebuilding the proximity graph.

6 Cross-edges Selection

In this section, we focus on the cross-edges selection problem (Definition 4.1). Let $Opt(G, S, M)$ denote the optimal minimum value of the maximum hop distance from a newly added node v to any node in S over G. We show the hardness of the problem in theory.

Theorem 6.1: *Given a graph G, a subset of nodes S in G, and an integer M, the cross-edges selection problem is NP-hard.*

Proof Sketch: We reduce the dominating set (DS) problem [14], which is NP-complete, to the cross-edges selection problem. Given a graph $G = (V, E)$ and an integer k, where $k < |V|$, the DS problem is to decide whether there exists a set D of k nodes such that any node of G is either in D or in a 1-hop neighbor of D. We can find that a graph has a dominating set with size k if and only if the cross-edges selection problem has $M = k$ and $Opt(G, S, M) = 2$. □

Consider the scale of the object dataset, $\mathcal{O}$, the proximity graph G and the subset of nodes, S, in G can be very large. We propose an approximate algorithm named SelectCrossEdge (Algorithm 6) with a theoretical guarantee. At a high level, our approximate algorithm solves the cross-edges selection problem with three main steps. ❶ We aim at adding some edges to ensure that the induced subgraph of G over the set of nodes S, forms a connected component. ❷ Given an integer α, the algorithm can ascertain if a solution exists where the maximum hop distance from v to any node in S is less than or equal to 2α. ❸ We aim at determining the minimum $\alpha_{\min}$ that yields a solution with a maximum hop distance less than or equal to $2\alpha_{\min}$, thereby offering a 2-approximate algorithm.

The First Step: Since the nodes that satisfy the predicate p may be anywhere in the proximity graph G, the induced subgraph of G over the set of nodes S can potentially contain many disconnected components. The number of such components can even exceed M, rendering the addition of M edges insufficient to address the connectivity issues. Therefore, the first step is to add some edges to ensure that the induced subgraph of G over the set of nodes S forms a single connected component.

To accomplish this, as shown in lines 1-7 in Algorithm 6, we use Q to store the subset of traversed nodes in S (line 1). We then proceed to iteratively select an untraversed node (i.e., from the set $S \setminus Q$) (line 3), initiating a depth-first search (DFS) from this node to find the set of nodes Q' from its connected component within the induced subgraph of G over the set of nodes S (line 4). For the newly discovered component from node u, we add an edge from a node in the traversed node set to it, such that all traversed nodes form a single connected component (lines 5-6), and update the set Q accordingly (line 7).

The Second Step: Given an integer α, the second step is presented in lines 11-17 in Algorithm 6.

Algorithm 6: SelectCrossEdge (G, S, M)

Input : a graph G, the node set S in G, M to limit the number of cross-edges
Output : the selected node set C

```
1  Q ← ∅;
2  while Q ≠ S do
3      Insert a randomly selected node u from S \ Q into Q;
4      Conduct a DFS from u on the induced subgraph of G over S, and mark the set of nodes traversed as
       Q';
5      if Q ≠ ∅ then
6          Add an edge from a randomly selected v ∈ Q to u;
7      Q ← Q ∪ Q';
8  l ← 0; r ← |S|;
9  while l ≤ r do
10     α ← ⌊(l+r)/2⌋;
11     S' ← ∅; V ← ∅;
12     while |V| < M do
13         if S = S' then break;
14         Insert the randomly selected node u in S \ S' into V;
15         Conduct a DFS from u on the induced subgraph of G over S, and mark the set of nodes from u
           within α-hop as Q';
16         S' ← S' ∪ Q';
17     if S' = S then C ← V; r ← α − 1;
18     else l ← α + 1;
19 return C;
```

Here, we use V to represent the set of nodes selected from S, where S' denotes the subset of S reachable by any node in V within an α-hop distance (line 11). This step involves iteratively selecting nodes from S until the size of V reaches M (lines 12-16). During each iteration, a node u is randomly selected from the set $S \setminus S'$, i.e., a node that cannot be reached by any node in V within an α-hop (line 14). The algorithm halts immediately if no such nodes exist (line 13). Next, u becomes a newly selected node to establish an edge from v to it, leading to an update of S' with node u through a depth-first search (DFS), containing nodes reachable from u within an α-hop (lines 15-16). Finally, a check is performed to verify if S' is equivalent to S (line 17). The following lemma shows the relationship between the optimal solution ($Opt(G, S, M)$) and a solution when $S' \neq S$ at line 17.

Lemma 6.1: *Given a graph G, a node set S in G, and an integer M, for a given integer α at line 10 in Algorithm 6, we have $Opt(G, S, M) > \lfloor \alpha/2 \rfloor + 1$ if $S' \neq S$ at line 17.*

Proof Sketch: For a given integer α, the nodes in V must satisfy that the hop distance between any pair of nodes in V is greater than α. Otherwise, assume that there exist two nodes x and y in V with a hop distance less than or equal to α, and x is added to V before y. Then, when x is selected at line 14, y would be added to S', leading to its absence in V later on, which is a contradiction.

Since $S' \neq S$, we conclude that $|V| = M$; and for any node u in $S' \setminus S$ at line 17, the hop distance from any node in V to u, and from u to them, exceeds α. Hence, we have $M + 1$ nodes in S where each pairwise hop distance is greater than α. Focusing solely on these $M + 1$ nodes in S, when

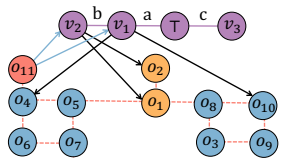


Fig. 5. An Example of Search Algorithm

adding edges from v to any M nodes in S, the hop distance from v to any of these $M+1$ nodes must exceed $\lfloor \frac{\alpha}{2} \rfloor + 1$. Therefore, we have $Opt(G, S, M) > \lfloor \frac{\alpha}{2} \rfloor + 1$. □

The Third Step: By using the same strategy for selecting a node in $S \setminus S'$ at line 14 in Algorithm 6, such as selecting the node with the smallest ID, a boundary, denoted as $\alpha_{\min}$, emerges that divides the values of α. That is, when $\alpha \geq \alpha_{\min}$, $S = S'$ at line 17 is satisfied, while for $\alpha < \alpha_{\min}$, $S' \neq S$ always holds. To quickly find this boundary, we employ binary search. As shown in lines 8-18 in Algorithm 6, initially, we set l as 0 and r as the size of node set S (line 8). Since the induced subgraph in G over the set of nodes S forms a connected component after processing with the first step, $S = S'$ at line 17 must hold when $\alpha = |S|$. Next, the binary search begins (line 9), at each time, the value of α is set as the mean of l and r. Through the second step, we verify if a solution exists where the maximum hop from a new node v to any nodes in S is $\alpha + 1$ (lines 11-16), and adjust the left and right boundaries accordingly (lines 17-18). Following the binary search, we obtain a set C corresponding to the minimum value of α where $S' = S$ at line 17, which equals the boundary $\alpha_{\min}$ as defined earlier.

The following theorem proves the approximation of our algorithm.

Theorem 6.2: *Given a graph G, a node set S in G, and an integer M, Algorithm 6 is a 2-approximate algorithm for the cross-edges selection problem.*

Proof Sketch: For a solution C returned from Algorithm 6, we denote its maximum hop from v to any node in S is d, which is equal to $\alpha_{\min} + 1$, where $\alpha_{\min}$ is the minimal value of α such that $S' = S$ at line 17. Based on Lemma 6.1, we have $Opt(G, S, M) > \lfloor \frac{\alpha_{\min}-1}{2} \rfloor + 1 = \lfloor \frac{d-2}{2} \rfloor + 1 = \lfloor \frac{d}{2} \rfloor \Rightarrow d < 2Opt(G, S, M)$. □

The first step in our algorithm requires $O(|G[S]|)$ time, where $|G[S]|$ represents the total number of nodes and edges in the induced subgraph of G over the set of nodes S. The next two steps of Algorithm 6 need $O(|G[S]| \log |S|)$ time. Hence, in total, the time complexity of Algorithm 6 is $O(|G[S]| \log |S|)$ time.

7 A New Search Algorithm

The new search algorithm is given in Algorithm 7. It first finds a minimal coverage set of nodes, M_p, in the AND/OR graph $G_\sharp$ for the query predicate c_q, and begins the search from the nodes in the proximity graph G that have edges from the nodes in M_p. Overall, the search process is similar to the beam search for k-ANN search, but with the following two distinctions.

(1) Search Utilizing Nodes in the AND/OR Graph: While the beam search solely on the proximity graph, G, our search algorithm utilizes nodes within the AND/OR graph $G_\sharp$. For a node u in our queue that does not satisfy the query predicate, c_q, if we find a cross-edge from node u to a node p_i in the AND/OR graph $G_\sharp$ that satisfies the predicate, it implies that u is one of the nodes with the maximum htops to nodes satisfy the predicate. We halt exploration of its proximity graph neighbors. Instead, we regard the nodes pointed to by cross-edges originating from p_i as the new neighbors to explore, if p_i has not yet been explored before.

Example 7.1: As shown in Fig. 5, there exist 11 objects with distinct categorical attributes. Among

Algorithm 7: Search ($G_+, q = (v_q, c_q), w$)

```
Input  : the enhanced proximity graph G+, a vector search query q = (v_q, c_q), and the beam width w
Output : k-ANN of v_q that satisfy predicate c_q
{p_1, ···, p_m} ← SearchGraph (G_♯, c_q); // Algorithm 2 or 4
Initialize two priority queues Q and S;
for each i ← 1 to m do
    Mark p_i visited;
    for each u where (p_i, u) in G+ do
        if u ∉ Q then push u into Q;
Q ← the w nodes in Q with minimum distance to v_q;
S ← the k nodes in Q with minimum distance to v_q;
while there exists an unvisited node in Q do
    u ← the nearest unvisited node to v_q in Q;
    Mark u visited;
    if u does not satisfy c_q then
        P ← the node set in G_♯ that satisfy c_q and are pointed by u;
    if u satisfies c_q or P = ∅ then
        for each v with edge (u, v) in the proximity graph G do
            if v is unvisited and v ∉ Q then
                Push v into Q;
                if v satisfies c_q then push v into S;
    else
        for each node p_i in P do
            if p_i is visited then continue else Mark p_i visited;
            for each v that the cross-edges from p_i point to do
                if v is unvisited and v ∉ Q then
                    Push v into Q; Push v into S;
    while |Q| > w do
        Remove the node u from Q with maximum distance to v_q;
    while |S| > k do
        Remove the node u from S with maximum distance to v_q;
return S;
```

these, $\{o_1, o_2\}$ have attribute $\{a, b\}$, $\{o_3, \cdots, o_{10}\}$ have attribute $\{a\}$, and $\{o_{11}\}$ has attribute $\{c\}$. The AND graph constructed for the categorical sets is depicted with purple nodes, and cross-edges are colored blue and black. We omit the cross-edges of v_3 here to focus on the issues with v_1 and v_2. Considering a query $q = (o_2, a)$, the search initially finds v_1, starting search at o_4 and o_{10}, both needs a 3-hop search to reach o_2 via beam search. In our algorithm, the exploration starts at o_4 and navigates to o_{11} since it is closer to o_2 than o_4. o_{11} does not satisfy the predicate, but it connects to v_2 by a cross-edge, where v_2 satisfies the predicate and remains unvisited. Thus, the search ignores the neighbors of o_{11}, opting to explore the neighbors of v_2 in the proximity graph, i.e., o_1 and o_2. This approach makes o_2 a 2-hop neighbor of o_4, thereby shortening the traversal path length.

(2) Two Searching Queues: In addition to a priority queue Q, which functions in the same way as

Table 3. Statistics of Datasets

Dataset	Dim.	#vectors	#queries	Vector type	Source	Dataset	Dim.	#vectors	#queries	Vector type	Source
SIFT	128	1,000,000	10,000	Image	[1]	Crawl	300	1,989,995	10,000	Text	[2]
GIST	960	1,000,000	1,000	Image	[1]	YFCC	192	10,000,000	100,000	Image	[37]
LAION	512	1,000,448	1,000	Image	[34]	Redcaps	512	1,000,000	1,000	Image & Text	[10]
GloVe	100	1,183,514	10,000	Text	[33]	arXiv	768	132,687	1,000	Text	[30]

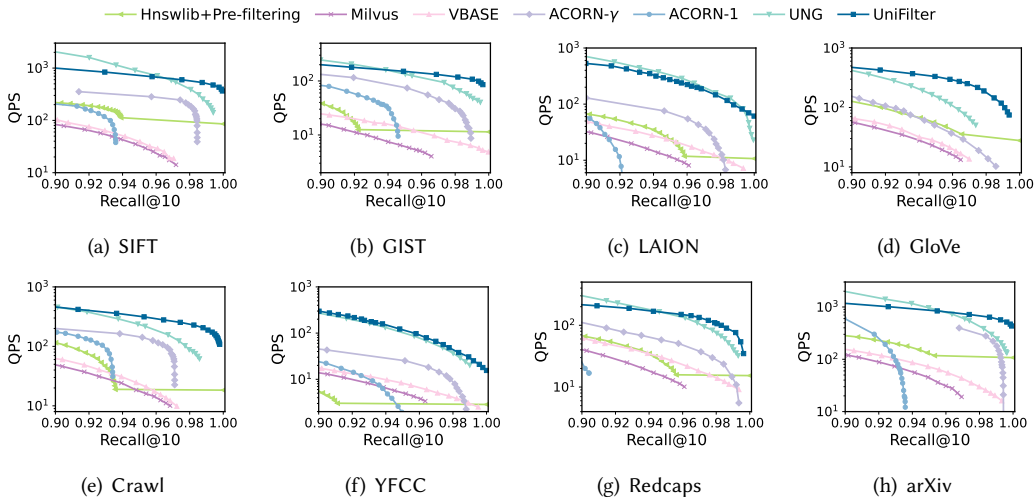
beam search by retaining the w nodes with the current smallest distance to the query, we introduce another queue S. S maintains the k nodes that not only satisfy the predicate but also possess the smallest current distance to the query vector. Notably, the nodes in S will be the final results when the search terminates. Note that, during search, a node traversed will be inserted into Q, to assist in navigating the nearest neighbors of the query, although it may not satisfy the predicate. To maintain Q to navigate, we can not omit its distance computation.

We present the details of our search algorithm in Algorithm 7. The algorithm begins by finding the minimal coverage set of nodes in the AND/OR graph (line 1). We initialize two queues, Q and S (line 2), which are the proximity queue for beam search and the queue for maintaining traversed nodes that satisfy the query predicate, respectively. To initiate the search on the enhanced proximity graph G_+, we start the search at nodes that have cross-edges from the nodes in the minimal coverage set (lines 3-8). Next, for each iteration, we select an unvisited node from Q that is closest to the query vector, v_q, and mark it as visited (lines 9-11). If the node u does not satisfy the query predicate, c_q, but has at least one cross-edge pointing to a node in the AND/OR graph that satisfies the query predicate (lines 12-13, 19), we halt its exploration in the proximity graph G. Instead, we utilize the neighbors of nodes pointed to by its cross-edges that satisfy the query predicate to resume exploration (lines 20-22), and update queues Q and S correspondingly (lines 23-24). Otherwise, we continue exploring the neighbors of u in the proximity graph (lines 14-18). Throughout the process, we ensure that the size of Q does not exceed the beam width w and that the size of S does not surpass k (lines 25-28). Ultimately, we return the nodes in S as the final results (line 29).

8 Experiments

Datasets: We employ 8 real-world public datasets along with their queries, which have been used in related research for vector search with both categorical attributes [6, 15] and numerical attributes [42, 46]. The datasets are summarized in Table 3. For Redcaps dataset, we generate a collection of one thousand query vectors by asking ChatGPT-4 to come up with queries for an image search system and embedding using CLIP. For arXiv dataset, query vectors are chosen randomly from the base vectors, whereas query vectors for the remaining datasets have been provided.

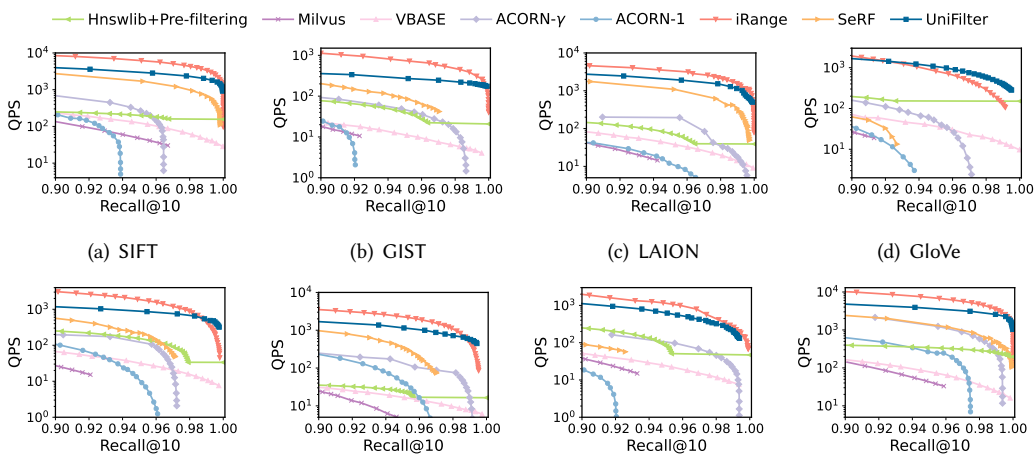
For each dataset, in addition to its vectors, we extract or generate predicates for evaluations. Specifically, for **categorical attributes**, we extract five attributes including "year", "month", "task", "method", and "dataset" from arXiv dataset, each of which has 12, 14, 1678, 890, and 1637 different values, respectively. For LAION dataset, following [31], we assign a keyword list to each vector by selecting the top 3 words with the highest text-to-image CLIP scores from a pool of 30 common adjectives and nouns. For YFCC dataset, following [35], each vector is linked to a tag list comprising words derived from the description, camera model, year of capture, and country. These tags are selected from a vocabulary of 200,386 potential tags. For other datasets, following the approach in [6, 31], we adopt Zipf distributions and directly use their source codes to obtain synthetic label sets for base vectors, in which contain 12 different labels (i.e., $|A_C| = 12$), resulting in a cardinality of $2^{12} - 1$. The query labels for all datasets are randomly generated according to the distribution of base labels. For **numerical attributes**, we extract timestamps from Redcaps dataset as the numerical attribute values of their vectors. For other datasets, followed [46], the numerical attribute values



(a) SIFT (b) GIST (c) LAION (d) GloVe

(e) Crawl (f) YFCC (g) Redcaps (h) arXiv

Fig. 6. Comparisons of Vector Search With Categorical Attributes Among Different Algorithms (Exp. 1)



(a) SIFT (b) GIST (c) LAION (d) GloVe

(e) Crawl (f) YFCC (g) Redcaps (h) arXiv

Fig. 7. Comparisons of Vector Search With Numerical Attributes Among Different Algorithms (Exp. 2)

are random integers from the range $[1, 10^4]$. For numerical range predicates of queries, we assign different query ranges to test the performance of different approaches under various workloads.

Algorithms: We conduct comparisons between our approach, named UniFilter, and the 6 state-of-the-art approaches, including single proximity graph approaches (Milvus [38], VBASE [45], ACORN [31]), multi-proximity graph approaches for categorical attributes (UNG [6]) and numerical attributes (Serf [46], iRange [42]). Moreover, we introduce a mixed filter strategy named Hnswlib+Pre-filtering as a baseline approach. For each algorithm, we use recommended parameters if provided, and try to tune other parameters on each dataset for a Pareto-optimal recall-QPS curve.

(1) Milvus [38] and VBASE [45] are both vector database systems that support generic attribute-filtering ANN queries. We choose HNSW as their index, and with grid search, we fix $EF = 400$, $M = 32$ for all datasets. (2) ACORN [31] is an index designed for general-filtered ANNS based on HNSW, offering two variants: ACORN-γ and ACORN-1. For ACORN-γ, [31] proposes to employ the pre-filtering method as a fallback for queries with specificity lower than $1/\gamma$. However, their official

code [3] does not implement the specificity estimator or this fallback strategy. Hence, we rely on the actual specificity to implement such a strategy. Following the parameters outlined in [31], we set $M = 32$ and $M_\beta = 64$ for all datasets. For ACORN-γ, we apply $\gamma = 100$ across all datasets in both label and range filter comparisons, considering that the smallest selectivity in our experiments is 1%. As for ACORN-1, aligning with [31], we utilize the same parameters as ACORN-γ, except for setting $\gamma = 1$ and $M_\beta = 2M$. (3) UNG [6] is a dedicated index method designed for queries with categorical attributes. In alignment with the parameters detailed in [6], we employ the Vamana algorithm [17] to construct its underlying proximity graphs, utilizing $\alpha = 1.2$, a degree bound of $R = 32$, and a maximum queue length of $L = 100$. We set $\delta = 6$ for constructing cross-group edges and also set $\delta = 6$ for query processing. (4) Serf [46] is a multi-proximity graph approach for queries with numerical attributes. It constructs dedicated graphs for all query ranges, and offers three index construction versions: MaxLeap, MidLeap, and MinLeap. We opt for the MaxLeap version in the comparison due to its showing optimal search performance with significantly smaller index time and space, as shown in [46]. We set $K = 100$ and $M = 64$ for index construction across all datasets. (5) iRange [42] is a dedicated index method for queries with numerical attributes. The parameter setting for its HNSW graph, specifically M and ef_{con}, aligns with that of Milvus and VBASE, with $M = 32$ and $ef_{con} = 400$ set across all datasets. (6) Hnswlib+Pre-filtering, we employ the codes from Hnswlib [26] for the post-filtering implementation with the same parameter setting to Milvus, integrating with the pre-filtering method. Specifically, we execute both of them, and the faster method will be chosen to illustrate in the QPS-recall curve for this approach. (7) UniFilter, our approach. In comparisons of both queries with categorical and numerical attributes, we establish the parameter M_p to be equivalent to the dataset size $|O|$. To ensure fair comparisons, since most of the above algorithms rely on the HNSW index, we adopt the identical HNSW index utilized for Milvus and VBASE as the proximity graph in our index for comparative evaluations.

Performance metrics: Following existing benchmarks [6, 42, 46], we employ QPS (Queries Per Second) to measure efficiency and use Recall to show the accuracy. QPS is the quantity of queries processed per second, while Recall is precisely defined in Section 2.

Experimental environments: The experiments are conducted on a CentOS Linux server with Intel(R) Xeon(R) Silver 4215 CPU @ 2.50GHz with 128GB memory. All algorithms are implemented in C++11. The code is compiled with g++ 11.4.1 under O3 optimization.

For index construction, we report the index time/size on one same single machine by utilizing multiple threads. For vector search, we set $k = 10$ and evaluate the performance of the search algorithms using a single thread following the previous works [31, 42, 46], and incrementally adjust parameters within the search algorithm to achieve the desired recall in the experiments.

Exp. 1. Overall performance of categorical queries. Fig. 6 shows the overall performance of vector search when querying with categorical attributes across all datasets. In this scenario, both the vectors in the dataset and the query vectors have varying numbers of labels, with dataset vectors satisfying the predicate conditions if they contain all the labels included in the query. The average query selectivity here is around 20%, indicating that, on average, 20% of the vectors satisfy the label predicates of the queries. The results demonstrate that on all datasets, our approach, UniFilter, remains highly efficient in contrast to the state-of-the-art single proximity graph approaches, i.e., Milvus, VBASE, ACORN-γ, and ACORN-1, and the baseline Hnswlib+Pre-filtering. Note that the curve of Hnswlib+Pre-filtering is not smooth at the last point, attributed to the shift in strategy from Hnswlib to pre-filtering. When compared with the state-of-the-art approach, UNG, our method mirrors its performance when the recall is low. However, as the recall increases, our QPS reduces at a much slower rate than UNG and even surpasses it at high recall levels. The superiority of our approach is particularly on the datasets where achieving high recall proves challenging, e.g., GloVe dataset. This advantage primarily stems from the larger size of the minimal coverage set (MCS)

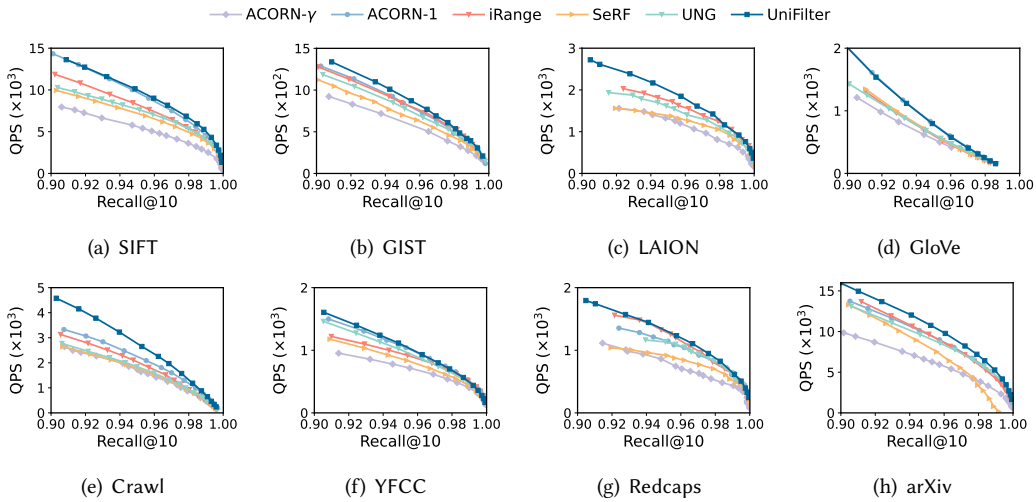


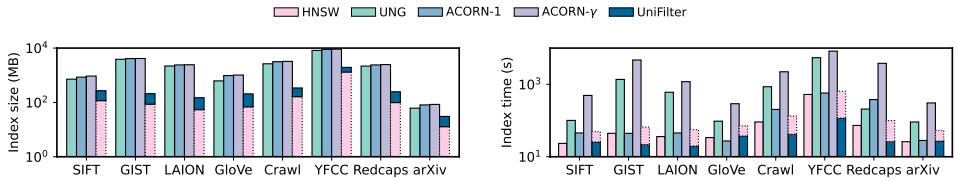
Fig. 8. Comparisons of Vector Search Without Predicates (Exp. 3)

required by UNG for each query. When the required recall is high, there are many subgraphs of the nodes in the MCS that need to be explored, making the accuracy of UNG improves slowly as the beam width widens. While for UniFilter, the time for finding an MCS occurs in a very small portion (less than 1%) of the total search cost for each query on all datasets, and the size of the MCS is bounded as $O(1)$.

Exp. 2: Overall performance of numerical queries. Fig. 7 presents the overall performance of queries with numerical attributes at a range coverage of 20%. Our approach, UniFilter, surpasses all single proximity graph approaches, i.e., Milvus, VBASE, ACORN-γ, ACORN-1, and Hnswlib+Pre-filtering on all datasets, while our approach shows significantly higher QPS for range-filtered vector search while maintaining the same recall levels. When compared to state-of-the-art approaches, Serf and iRange, Serf shows suboptimal query performance across all datasets, primarily due to its heavy compression. Although iRange marginally outperforms UniFilter in most cases across all datasets, our approach retains advantages, particularly at high recall levels, as shown in datasets like GloVe and Crawl. Moreover, in UniFilter, the average time across all datasets for finding the minimal coverage set (Algorithm 4) is less than $1\mu s$ (much less than 1% of the total query time).

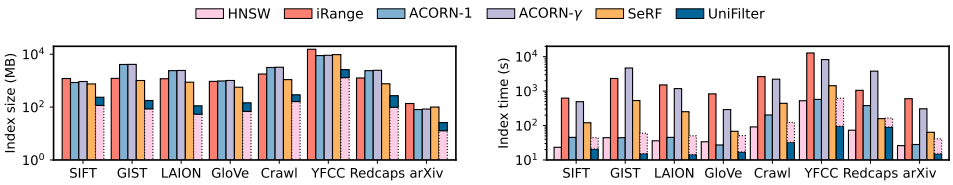
Exp. 3: Search performance of vector search without predicates. In this set of experiments, we compare our approach with others when querying without predicates, i.e., k-ANN search. Since Milvus and VBASE utilize the same HNSW index as ours, their search performance is the same as ours, thus, they are excluded here. The results shown in Fig. 8 consistently show that our method outperforms others in k-ANN search, as we do not compromise the index of the proximity graph. For iRange, its suboptimal performance is mainly because of its reliance on NSW rather than HNSW in its implementation, while Serf and UNG compromise the index structure for k-ANN search due to their construction algorithms. For ACORN, ACORN-1 demonstrates similar search performance to ours, as it merely involves adding some nodes in the traversal. Conversely, ACORN-γ exhibits the poorest performance since it introduces much more nodes in the index, thereby increasing the degree for each node and consequently reducing search efficiency.

Exp. 4: Index comparisons of vector search on categorical attributes. Fig. 9(a) displays the memory usage of all methods across various datasets. Note that in the comparisons of Exp. 1, Milvus, and VBASE use the same HNSW index, hence, we report the HNSW index size. Our approach is also built upon the same HNSW index, where the graph of HNSW is part of our index structure.



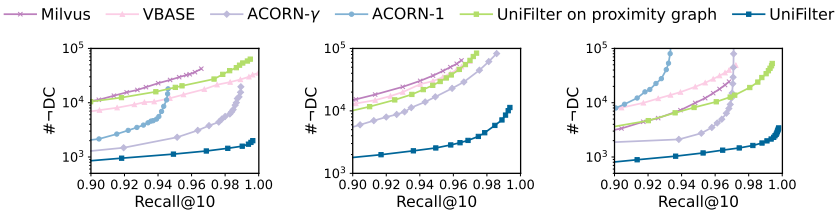
(a) Size of Graph Topology (b) Index Time

Fig. 9. Indexes for Categorical Attributes (Exp. 4)



(a) Size of Graph Topology (b) Index Time

Fig. 10. Indexes for Numerical Attributes (Exp. 5)



(a) GIST (b) GloVe (c) Crawl

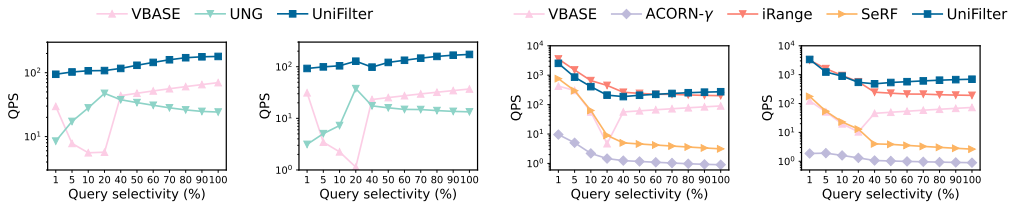
Fig. 11. Comparisons of Distance Computations (Exp. 6)

Therefore, for our index size, we present it alongside the size of HNSW to illustrate the additional size in our index compared to HNSW. Among these methods, ACORN-γ has the largest index size due to its extension of the neighbor list size of each node in the graph. ACORN-1 is slightly smaller than ACORN-γ but still larger than UNG in size, although UNG outperforms ACORN-1 in search performance. For our index size, it slightly larger than the original HNSW size but considerably smaller than the other approaches.

For indexing time, Fig. 9(b) shows the indexing times for various approaches across all datasets. Our indexing time is significantly smaller than that of other approaches, even smaller than HNSW. This advantage stems from our index being constructed based on the given proximity graph, resulting in substantially reduced distance computations during indexing compared to other approaches.

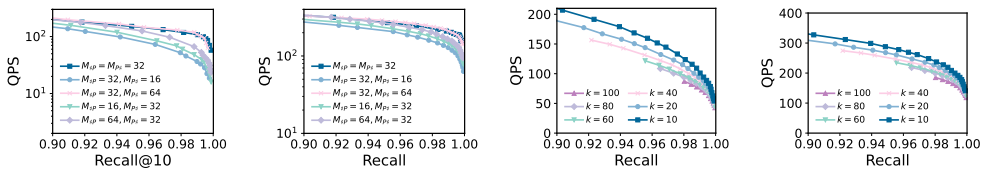
Exp. 5: Indexing comparisons of vector search on numerical attributes. Fig. 10 presents the memory usage and construction time of various approaches. Both Milvus and VBASE utilize the same HNSW for their indexing. As HNSW is part of our index, its memory size is reported with our total memory usage. The results show that while single proximity graph approaches need smaller memory usage, they often suffer from poor search performance. Serf has less memory overhead than iRange, attributed to its aggressive compression strategy, but it compromises in performance and query precision. Our approach consistently excels in index building time, requiring only marginally more memory than HNSW.

Exp. 6: Distance computations of vector search with predicates. We report the number of distance computations for vectors that do not satisfy the query predicate, denoted as #¬DC, when



(a) GIST (cate.) (b) Crawl (cate.) (c) GIST (nume.) (d) Crawl (nume.)

Fig. 12. Varying Attribute Selectivity (Exp. 7)



(a) Vary M (cate.) (b) Vary M (nume.) (c) Vary k (cate.) (d) Vary k (nume.)

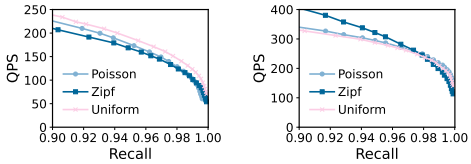
Fig. 13. Impact of Parameter Values on GIST (Exp. 8)

employing various search methods across GIST, GloVe and Crawl datasets. We use the queries on categorical attributes as an example here for testing. Multi-proximity graph approaches are designed to avoid traversing nodes that do not satisfy the predicate, hence, they are not included in this experiment. Fig. 11 shows that the number of vectors not satisfying the query predicate traversed in UniFilter is notably lower than other approaches. Moreover, comparing our method to ones utilizing our search algorithm directly on the proximity graph reveals the enhancements made by our index structure, which limits the traversal to nodes that do not satisfy the predicate.

Exp. 7: Search performance when varying the query selectivity. In this set of experiments, we examine the impact of query selectivity on query performance. By modifying the query selectivity through the random generation of categorical and numerical sets, we adjust the query selectivity values from 1% to 100%. In Fig. 12, we present QPS of different approaches on GIST and Crawl datasets when achieving a recall@10 of 0.99. Milvus, ACORN-γ, and ACORN-1 are excluded from comparisons on categorical attributes, and ACORN-1 and Milvus are excluded from comparisons on numerical attributes, due to their QPS for achieving recall@10=0.99 is below 1. The results show that our performance surpasses others while maintaining a comparable level to iRange, and even outperforms iRange when the selectivity is high. Note that VBASE has a rapid QPS shift at around 40% selectivity. It is because the incorporation of the pre-filtering strategy in VBASE, results in its high efficiency for queries with high selectivity. All multi-proximity graph methods exhibit a gradual decline in QPS as selectivity increases, but UNG initially has an increase in QPS, due to the number of retrieved proximity graphs decreasing initially for UNG.

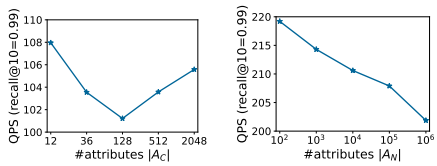
Exp. 8: Impact of parameters. Fig. 13 shows the results on GIST while varying M and k. For M, $M_{\sharp P}$ and $M_{P\sharp}$ represent the degree limits on $\sharp P$- and $P\sharp$-edges respectively. The search performance initially improves with increasing $M_{\sharp P}$, then declines. It shows a larger improvement with increasing $M_{P\sharp}$ but stabilizes with a slight enhancement at higher values of $M_{P\sharp}$. For k, UniFilter consistently exhibits high query efficiency and accuracy across various k values. As k increases from 10 to 100, a gradual decline in query performance is noticeable, mainly due to the increased number of candidates that must be traversed during the search process.

Exp. 9: Varying attributes distribution. To assess the robustness of UniFilter across various



(a) Categorical (b) Numerical

Fig. 14. Vary Attribute Distribution on GIST (Exp. 9)



(a) Categorical (b) Numerical

Fig. 15. Vary Size of Attributes on GIST (Exp. 10)

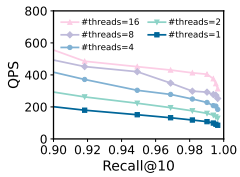


Fig. 16. Varying number of threads (Exp. 11)

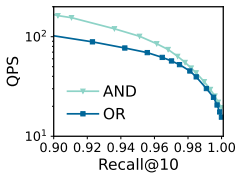
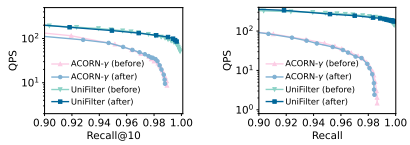


Fig. 17. Multiple Attribute Query (Exp. 12)



(a) Categorical (b) Numerical

Fig. 18. Search Performance Before and After Updates (Exp. 13)

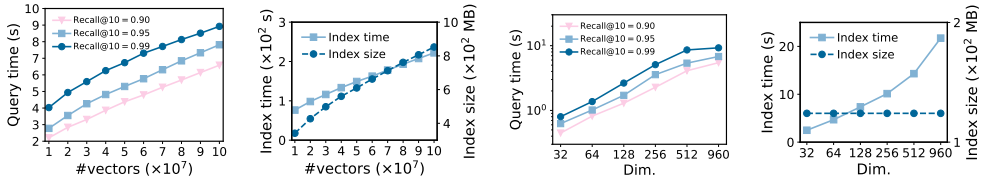
attribute distributions, we employ three distributions to generate categorical and numerical attributes on GIST dataset. As depicted in Fig. 15, UniFilter consistently demonstrates high efficiency and maintains robust performance to attain high recall rates across diverse distributions.

Exp. 10: Search performance when varying the size of attributes. Using GIST dataset as an example, we report the search performance when vary the attribute sizes. Ranging the value of $|A_C|$ from 12 to 2048, where $|A_C|$ leads to a cardinality of $2^{|A_C|} - 1$, we find that the QPS remains stable and even improves as $|A_C|$ increases, attributed to the merge strategy in AND graph. We also adjust the value of $|A_N|$ from 10^2 to 10^6, QPS shows a slight decrease as the number of numerical attributes significantly rises.

Exp. 11: Query processing with multiple threads. In the parallel implementation of UniFilter, each query is treated as an atomic operation and processed in parallel. We vary the number of threads from $\{1, 2, 4, 8, 16\}$ on GIST dataset for categorical attributes. The results, depicted in Figure 16, illustrate that search efficiency improves with an increase in the number of threads, indicating the robustness of UniFilter across various thread counts.

Exp. 12: Queries with multiple attributes. UniFilter is capable of handling scenarios where an object is associated with multiple categorical/numerical attributes, and a query predicate can be specified by a combination of them, incorporating AND or OR semantics. To support such cases, a combined AND/OR graph can be employed. In such a graph, edges are categorized into two types: (1) AND-edge from u to v with label $L_{u,v}$, where $\text{ext}(v) = \text{ext}(u) \cap O_{L_{u,v}}$; and (2) OR-edge from u to v with label $L_{u,v}$, where $\text{ext}(v) = \text{ext}(u) \cup O_{L_{u,v}}$. To construct this combined AND/OR graph, objects are inserted by adding nodes representing their multiple attributes individually. Nodes representing the same set of objects are merged. Given a query predicate, traversal can start from node $\top$ or $\bot$, navigating through the edges in the combined AND/OR graph to identify nodes that represent the equivalent predicates. Fig. 17 depicts the search performance when each object in GIST dataset contains at most 12 categorical attributes and 1 numerical attribute. Queries involve AND/OR semantics between categorical and numerical attributes, i.e., $c_q = \{l_1, l_2, \cdots\} \wedge [q_l, q_r]$ or $c_q = \{l_1, l_2, \cdots\} \vee [q_l, q_r]$. The average selectivity of queries is around 20%. Here, the index size is 169.9MB (including 85.8MB HNSW), and the time required for index building is 25.6+(44.3)s.

Exp. 13: Dealing with data updates. Since all existing multi-proximity graph approaches for



(a) Vary #vectors (b) Vary #vectors (c) Vary dim. (d) Vary dim.

Fig. 19. Scalability (Exp. 14)

vector search with predicates do not support updating, we compare our approach with single-proximity graph methods in managing updates on both numerical and categorical attributes, using ACORN-γ as an example. We utilize the state-of-the-art method [36] for proximity graph maintenance. To compare approaches, we initially remove 10% of objects from GIST datasets, followed by their reinsertion. The entire process takes ACORN-γ and UniFilter 906.187s and 948.952s on categorical attributes, and 906.187s and 960.560s on numerical attributes. Compared to ACORN-γ, UniFilter needs additional time for maintaining the AND/OR graph and cross-edges. Fig. 18 presents the performance of UniFilter and ACORN-γ before and after the updates, showcasing nearly identical results on search performance. We further evaluate attributes updating solely, where the proximity graph remains unchanged. We use the same label generator for GIST dataset and change attributes for 10% randomly selected objects; the updating process requires 42.08 seconds, with comparable search performance compared to before the update.

Exp. 14: Scalability. We evaluate the scalability of our approach on SIFT100M dataset by increasing the vector count from 10M to 100M. In Fig. 19(a)(b), we present the average query processing times for recalls of 0.90, 0.95, and 0.99. As the dataset size increases, the average query time, index time, and index size exhibit a linear increase while the size of the dataset increases, demonstrating the scalability of our methods. Furthermore, we assess scalability by varying the dimensions of datasets. Using GIST dataset, we vary the number of dimensions (i.e., the vector prefixes of specific dimension numbers) across $\{32, 64, 128, 256, 512, 960\}$. Fig. 19(c)(d) illustrates the query time, index time, and index size as dimensions vary, showcasing scalability with dimension increases. Note that the index size remains constant since we use identical parameters for index construction, the dimension will not affect the index size.

9 Conclusion

In this paper, we present a new all-in-one approach to support k-ANN query with and without predicates over large high-dimensional datasets. Our extensive experiments demonstrate the advantages of our approach regarding the accuracy and efficiency. First, for the cases without predicates, our approach upholds equivalent search performance to the state-of-the-art methods for k-ANN queries. Second, for the cases with predicates, our approach outperforms all state-of-the-art single proximity graph approaches, demonstrating an efficiency improvement up to 1 order of magnitude. In comparison with multi-proximity graph approaches, at high recall levels, e.g., recall@10=0.99, our search performance is 14.0x faster than UNG and 3.1x faster than iRange. In terms of index size and building time, on average, our index size is 6.7x smaller and index building time is 5.3x faster than that of UNG, and our index size is 10.7x smaller and index building time is 32.5x faster than that of iRange.

Acknowledgements

This work was supported by the National Key S&T Special Projects of China, No. 2025ZD1200600.

References

[1] 2010. Datasets for approximate nearest neighbor search. http://corpus-texmex.irisa.fr/.

[2] 2023. Common Crawl. https://commoncrawl.org/.

[3] 2024. ACORN Source Code. https://github.com/stanford-futuredata/ACORN.

[4] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.* 87 (2020).

[5] Thierry Bertin-Mahieux, Daniel P. W. Ellis, Brian Whitman, and Paul Lamere. 2011. The Million Song Dataset. In *ISMIR 2011*. University of Miami, 591–596.

[6] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024), 246:1–246:27.

[7] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *NeurIPS 2021*. 5199–5212.

[8] Mark de Berg. 1997. *Computational geometry: algorithms and applications*. Springer.

[9] DW Dearholt, N Gonzales, and G Kurup. 1988. Monotonic search networks for computer vision databases. In *Twenty-Second Asilomar Conference on Signals, Systems and Computers*, Vol. 2. IEEE, 548–553.

[10] Karan Desai, Gaurav Kaul, Zubin Aysola, and Justin Johnson. 2021. RedCaps: Web-curated image-text data created by the people, for the people. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021*.

[11] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net.

[12] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 8 (2022), 4139–4150.

[13] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[14] Michael R Garey and David S Johnson. 2002. *Computers and intractability*. Vol. 29. wh freeman New York.

[15] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. ACM, 3406–3416.

[16] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016*. IEEE Computer Society, 5713–5722.

[17] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc.

[18] Govinda D. Kurup. 1992. *Database Organized on the Basis of Similarities with Applications in Computer Vision*. Ph. D. Dissertation.

[19] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.

[20] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020*. ACM, 2539–2554.

[21] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement. *IEEE Trans. Knowl. Data Eng.* 32, 8 (2020), 1475–1488.

[22] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *CoRR* abs/2412.02448 (2024).

[23] Ying Liu, Dengsheng Zhang, Guojun Lu, and Wei-Ying Ma. 2007. A survey of content-based image retrieval with high-level semantics. *Pattern Recognit.* 40, 1 (2007), 262–282.

[24] Yingfan Liu, Yandi Zhang, Jiadong Xie, Hui Li, Jeffrey Xu Yu, and Jiangtao Cui. 2025. Privacy-Preserving Approximate Nearest Neighbor Search on High-Dimensional Data. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, 3017–3029.

[25] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Inf. Syst.* 45 (2014), 61–68.

[26] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using

Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[27] Tomás Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *Advances in Neural Information Processing Systems*. 3111–3119.

[28] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-Throughput Vector Similarity Search in Knowledge Graphs. *Proc. ACM Manag. Data* 1, 2 (2023), 197:1–197:25.

[29] Nasser M Nasrabadi and Robert A King. 1988. Image coding using vector quantization: A review. *IEEE Transactions on communications* 36, 8 (1988), 957–971.

[30] Malte Ostendorff, Till Blume, Terry Ruas, Bela Gipp, and Georg Rehm. 2022. Specialized document embeddings for aspect-based similarity of research papers. In *JCDL '22: The ACM/IEEE Joint Conference on Digital Libraries in 2022*. ACM, 7.

[31] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3 (2024), 120.

[32] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1 (2023), 54:1–54:27.

[33] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, A meeting of SIGDAT, a Special Interest Group of the ACL*. ACL, 1532–1543.

[34] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. 2021. LAION-400M: Open Dataset of CLIP-Filtered 400 Million Image-Text Pairs. *CoRR* abs/2111.02114 (2021).

[35] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, Dmitry Baranchuk, Edo Liberty, Frank Liu, Benjamin Landrum, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Yizhuo Chen, Weiguo Zheng, Zihao Wan, Jie Yin, and Ben Huang. 2024. Results of the Big ANN: NeurIPS'23 competition. *CoRR* abs/2409.17424 (2024).

[36] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *CoRR* abs/2105.09613 (2021).

[37] Bart Thomee, David A. Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. 2016. YFCC100M: the new data in multimedia research. *Commun. ACM* 59, 2 (2016), 64–73.

[38] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *SIGMOD '21*. ACM, 2614–2627.

[39] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978.

[40] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proc. VLDB Endow.* 13, 12 (2020), 3152–3165.

[41] Jiadong Xie, Jeffrey Xu Yu, and Yingfan Liu. 2025. Graph Based K-Nearest Neighbor Search Revisited. *ACM Trans. Database Syst.* 50, 4, Article 14 (June 2025), 30 pages.

[42] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6 (2024), 239:1–239:26.

[43] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 6 (2025), 1825–1838.

[44] Chaoqun Zhan, Maomeng Su, Chuangxian Wei, Xiaoqiang Peng, Liang Lin, Sheng Wang, Zhe Chen, Feifei Li, Yue Pan, Fang Zheng, and Chengliang Chai. 2019. AnalyticDB: Real-time OLAP Database System at Alibaba Cloud. *Proc. VLDB Endow.* 12, 12 (2019), 2059–2070.

[45] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2023*. USENIX Association, 377–395.

[46] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024), 69:1–69:26.

Received April 2025; revised July 2025; accepted August 2025